

KairosCRM

Prompt de arquitetura para o Claude Code — MVP v1.0

Este documento contém o prompt estruturado para o Claude Code construir o MVP do KairosCRM: plataforma SaaS de gerenciamento de mensagens e automação com IA para WhatsApp (Evolution API) e Instagram (Meta Cloud API).

Stack obrigatória

- Backend:** Flask (Python) com Flask-SocketIO para real-time
- Frontend:** Next.js 14 (App Router) com TypeScript e Tailwind CSS
- Banco de dados:** PostgreSQL com SQLAlchemy ORM e Alembic para migrations
- Cache e filas:** Redis com RQ ou Celery
- Editor de fluxo:** React Flow (drag and drop do agente autônomo)
- Autenticação:** JWT com refresh token

Arquitetura do projeto

Estrutura de pastas:

```
kairos-crm/  
  backend/  
    app/  
      __init__.py  
      config.py  
      extensions.py      # db, redis, socketio, celery  
      models/  
        user.py  
        workspace.py  
        contact.py  
        conversation.py  
        message.py  
        agent.py  
        flow.py  
      routes/  
        auth.py  
        conversations.py  
        messages.py  
        agents.py  
        flows.py  
        webhooks/  
          instagram.py  
          whatsapp.py  
        settings.py  
        services/
```

```

■ ■ ■ ■ ■ instagram_service.py
■ ■ ■ ■ ■ whatsapp_service.py
■ ■ ■ ■ ■ ai_agent_service.py
■ ■ ■ ■ ■ flow_engine.py
■ ■ ■ ■ ■ queue_service.py
■ ■ ■ ■ ■ tasks/
■ ■ ■ ■ ■ process_message.py
■ ■ ■ ■ ■ ai_response.py
■ ■ ■ ■ ■ migrations/
■ ■ ■ ■ ■ requirements.txt
■ ■ ■ ■ ■ run.py
■ ■ ■ ■ ■ frontend/
■ ■ ■ ■ ■ app/
■ ■ ■ ■ ■ (auth)/
■ ■ ■ ■ ■ login/page.tsx
■ ■ ■ ■ ■ register/page.tsx
■ ■ ■ ■ ■ (dashboard)/
■ ■ ■ ■ ■ layout.tsx
■ ■ ■ ■ ■ conversations/
■ ■ ■ ■ ■ page.tsx
■ ■ ■ ■ ■ [id]/page.tsx
■ ■ ■ ■ ■ agents/
■ ■ ■ ■ ■ page.tsx
■ ■ ■ ■ ■ [id]/editor/page.tsx
■ ■ ■ ■ ■ settings/page.tsx
■ ■ ■ ■ ■ components/
■ ■ ■ ■ ■ chat/
■ ■ ■ ■ ■ ConversationList.tsx
■ ■ ■ ■ ■ ChatWindow.tsx
■ ■ ■ ■ ■ MessageBubble.tsx
■ ■ ■ ■ ■ MessageInput.tsx
■ ■ ■ ■ ■ flow/
■ ■ ■ ■ ■ FlowEditor.tsx
■ ■ ■ ■ ■ nodes/
■ ■ ■ ■ ■ TriggerNode.tsx
■ ■ ■ ■ ■ MessageNode.tsx
■ ■ ■ ■ ■ ConditionNode.tsx
■ ■ ■ ■ ■ AINode.tsx
■ ■ ■ ■ ■ WebhookNode.tsx
■ ■ ■ ■ ■ hooks/
■ ■ ■ ■ ■ lib/

```

Banco de dados — modelos principais

Crie as seguintes tabelas com SQLAlchemy:

users: id, email, password_hash, name, created_at

workspaces: id, name, owner_id (FK users), plan, created_at

workspace_members: workspace_id, user_id, role

integrations: id, workspace_id, channel (instagram|whatsapp), status (active|inactive), credentials (JSON criptografado), meta (JSON)

contacts: id, workspace_id, channel, external_id, name, avatar_url, metadata (JSON), created_at

conversations: id, workspace_id, contact_id, channel, status (open|closed|bot), last_message_at, assigned_to (FK users), ai_enabled (bool), created_at

messages: id, conversation_id, direction (inbound|outbound), content, content_type, status (sent|delivered|read|failed), external_id, created_at

agents: id, workspace_id, name, system_prompt, model (claude-sonnet-4-20250514), temperature, enabled (bool), channels (JSON array), created_at

flows: id, agent_id, name, trigger_type, trigger_config (JSON), nodes (JSON), edges (JSON), active (bool), created_at

Integrações

WhatsApp — Evolution API

Base URL configurável por workspace. Endpoints consumidos pelo backend:

POST /message/sendText/{instance} — enviar texto

POST /message/sendMedia/{instance} — enviar mídia

Payload do webhook recebido em POST /webhooks/whatsapp:

```
{
  "event": "messages.upsert",
  "instance": "nome-instancia",
  "data": {
    "key": {
      "remoteJid": "556599999999@s.whatsapp.net",
      "fromMe": false,
      "id": "MSG_ID"
    },
    "message": { "conversation": "texto da mensagem" },
    "messageTimestamp": 1234567890,
    "pushName": "Nome do contato"
  }
}
```

Extrair número removendo @s.whatsapp.net. Criar ou localizar contato e conversa. Salvar mensagem. Enfileirar no Redis para IA se ai_enabled = true.

Instagram — Meta Cloud API

Base: <https://graph.facebook.com/v19.0/> — Envio: POST /{ig-user-id}/messages com Authorization: Bearer {page_access_token}

Verificação do webhook (GET /webhooks/instagram): validar hub.verify_token e retornar hub.challenge.

Payload do webhook recebido em POST /webhooks/instagram:

```
{
  "object": "instagram",
  "entry": [{
    "id": "IG_USER_ID",
    "messaging": [{
      "sender": { "id": "SENDER_IGSID" },
      "recipient": { "id": "RECIPIENT_IGSID" },
      "timestamp": 1234567890,
      "message": { "mid": "MSG_ID", "text": "texto" }
    }]
  }]
}
```

Fila de processamento (Redis + Celery/RQ)

1. Webhook recebe mensagem → salva no banco → retorna 200 imediatamente.
2. Enfileira task `process_message(message_id)`.
3. Task verifica se conversa tem `ai_enabled = true` e agente ativo para o canal.
4. Se sim: monta histórico das últimas 20 mensagens → chama Claude API → salva resposta → envia via canal → salva mensagem outbound.
5. Se não: emite evento via SocketIO para o frontend atualizar.

Agente de IA

- Modelo: `claude-sonnet-4-20250514`
- System prompt configurável por agente.
- Contexto: histórico das últimas 20 mensagens da conversa.
- Canais: cada agente tem lista de canais ativos (whatsapp, instagram ou ambos).
- Só responde se o canal da conversa estiver na lista e `ai_enabled = true`.

Endpoints de controle:

PATCH /api/agents/{id} — campos: `enabled`, `channels`

PATCH /api/conversations/{id}/ai — campo: `ai_enabled`

Editor de fluxo (React Flow)

Nodes disponíveis no MVP:

TriggerNode: gatilho — primeira mensagem, palavra-chave, horário

MessageNode: enviar mensagem de texto fixo

ConditionNode: if/else baseado em conteúdo ou tag do contato

AINode: acionar agente com prompt customizado para esse passo

WebhookNode: chamar URL externa via POST com payload configurável

O JSON de nodes e edges é salvo diretamente em `flows.nodes` e `flows.edges`. O `flow_engine.py` interpreta esse JSON em runtime. O editor tem sidebar de nodes, panel de configuração ao clicar, auto-save com debounce de 2s e botão ativar/desativar.

Frontend — telas principais

/conversations

Layout três colunas: filtros | lista de conversas | chat. A lista mostra ícone do canal, nome, última mensagem, timestamp e badge de IA ativa. Chat com scroll infinito, input com envio por Enter, toggle de IA por conversa.

/agents

Lista de agentes com toggle global de ativar/desativar e chips por canal (WhatsApp, Instagram). Botão para abrir o editor de fluxo.

/agents/[id]/editor

Tela cheia com React Flow. Sidebar esquerda com nodes, canvas central, panel direita com config do node selecionado. Header com status e botão salvar.

Real-time (SocketIO)

Eventos emitidos pelo backend:

new_message: { conversation_id, message } — nova mensagem recebida

conversation_updated: { conversation_id, fields } — status, ai_enabled etc.

agent_response_sent: { conversation_id, message } — resposta da IA enviada

Variáveis de ambiente

Backend (.env):

```
DATABASE_URL=postgresql://user:pass@localhost/kairos_crm
REDIS_URL=redis://localhost:6379/0
ANTHROPIC_API_KEY=
SECRET_KEY=
EVOLUTION_API_URL=
EVOLUTION_API_KEY=
META_VERIFY_TOKEN=
```

Frontend (.env.local):

```
NEXT_PUBLIC_API_URL=http://localhost:5000
NEXT_PUBLIC_SOCKET_URL=http://localhost:5000
```

Regras de implementação

1. Toda rota de API (exceto /webhooks e /auth) requer JWT válido no header Authorization: Bearer {token}.
2. Multi-tenant: toda query filtra por workspace_id. Nunca retornar dados de outro workspace.
3. Webhooks respondem 200 imediatamente e processam de forma assíncrona via fila.
4. Credentials das integrações ficam criptografadas no banco com Fernet (cryptography lib).
5. Migrations com Alembic — nunca alterar tabela diretamente.
6. Frontend usa SWR para fetching com revalidação automática.
7. Erros da API retornam sempre { error: string, code: string } com HTTP status adequado.
8. Logs estruturados (JSON) no backend com nível de log configurável via ENV.

Ordem de implementação

- 01.** Setup do projeto (estrutura de pastas, dependências, config)
- 02.** Models e migrations do banco de dados
- 03.** Autenticação JWT (register, login, refresh)
- 04.** Webhook WhatsApp (receber, parsear, salvar)
- 05.** Webhook Instagram (verificação + receber, parsear, salvar)
- 06.** Serviço de envio WhatsApp via Evolution API
- 07.** Serviço de envio Instagram via Graph API
- 08.** Fila Redis + task de processamento de mensagem
- 09.** Integração com Claude API no agente
- 10.** Rotas REST de conversations e messages
- 11.** Rotas REST de agents e flows
- 12.** SocketIO real-time
- 13.** Frontend: autenticação
- 14.** Frontend: tela de conversations com chat
- 15.** Frontend: tela de agents com toggles
- 16.** Frontend: editor de fluxo com React Flow
- 17.** Docker Compose para desenvolvimento local

Comece pelo passo 1. Pergunte se tiver dúvida sobre alguma decisão de arquitetura antes de implementar, não assuma.